

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset :

```
df = pd.read_csv("/content/sample_data/student_dataset_10000_rows.csv")
```

Data Analysis :

```
df.shape
```

```
(10000, 8)
```

```
df.columns
```

```
Index(['study_hours', 'attendance', 'sleep_hours', 'internet_usage',
       'assignments_completed', 'previous_score', 'exam_score',
       'placement_status'],
      dtype='object')
```

```
df.head(10)
```

	study_hours	attendance	sleep_hours	internet_usage	assignments_completed
0	7	56	8	7	
1	4	69	5	3	
2	11	60	7	6	
3	8	99	9	8	
4	5	52	8	6	
5	7	40	9	10	
6	10	40	4	11	
7	3	86	7	3	
8	7	72	8	5	
9	11	77	4	9	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   study_hours                          10000 non-null  int64
1   attendance                          10000 non-null  int64
2   sleep_hours                         10000 non-null  int64
3   internet_usage                      10000 non-null  int64
4   assignments_completed               10000 non-null  int64
5   previous_score                     10000 non-null  int64
6   exam_score                         10000 non-null  float64
7   placement_status                   10000 non-null  object
dtypes: float64(1), int64(6), object(1)
memory usage: 625.1+ KB
```

```
df.describe()
```

	study_hours	attendance	sleep_hours	internet_usage	assignments
count	10000.000000	10000.000000	10000.000000	10000.000000	1
mean	5.989600	69.88460	6.498500	6.062600	
std	3.163589	17.61653	1.709354	3.138163	
min	1.000000	40.00000	4.000000	1.000000	
25%	3.000000	55.00000	5.000000	3.000000	
50%	6.000000	70.00000	6.500000	6.000000	
75%	9.000000	85.00000	8.000000	9.000000	
max	11.000000	100.00000	9.000000	11.000000	

```
df.rename(columns={'study_hours' : 'study hrs' } , inplace = True)
```

```
df.columns
```

```
Index(['study hrs', 'attendance', 'sleep_hours', 'internet_usage',
      'assignments_completed', 'previous_score', 'exam_score',
      'placement_status'],
      dtype='object')
```

```
df.columns
```

```
Index(['study hrs', 'attendance', 'sleep_hours', 'internet_usage',
      'assignments_completed', 'previous_score', 'exam_score'],
      dtype='object')
```

```
pd.isnull(df).sum()
```

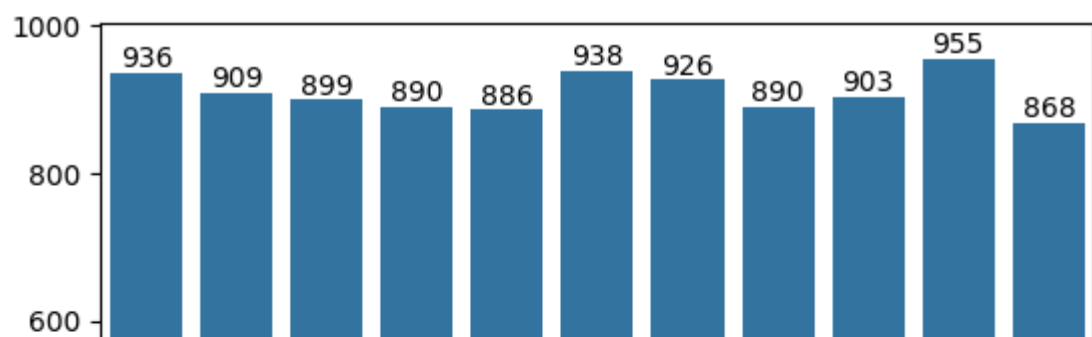
	0
study hrs	0
attendance	0
sleep_hours	0
internet_usage	0
assignments_completed	0
previous_score	0
exam_score	0

dtype: int64

EDA(EXPLORATORY DATA ANALYSIS):

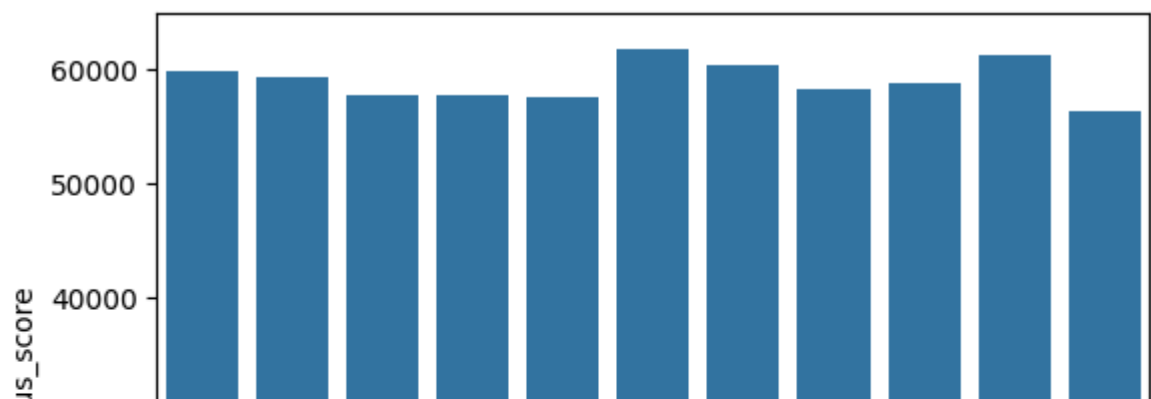
Study hours :

```
ax = sns.countplot(data = df, x = 'study hrs')
for bars in ax.containers:
    ax.bar_label(bars)
```



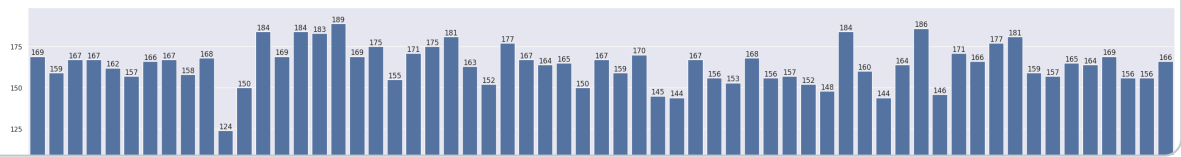
```
perf_study = df.groupby(['study hrs'], as_index = False)['previous_score'].sum()
sns.barplot(x='study hrs', y='previous_score', data = perf_study)
```

<Axes: xlabel='study hrs', ylabel='previous_score'>



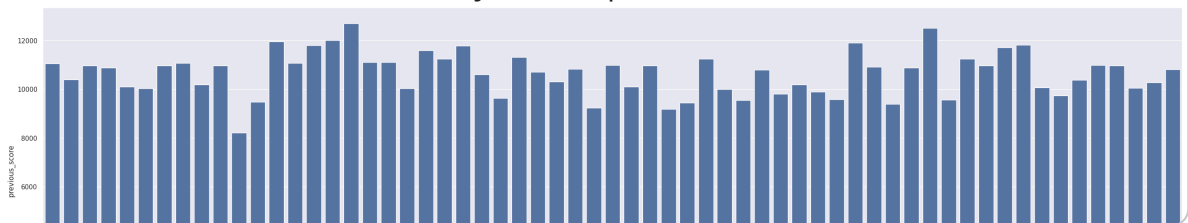
Attendance :

```
sns.set(rc = {'figure.figsize' : (35,10)})
ax = sns.countplot(data = df, x = 'attendance')
for bars in ax.containers:
    ax.bar_label(bars)
```



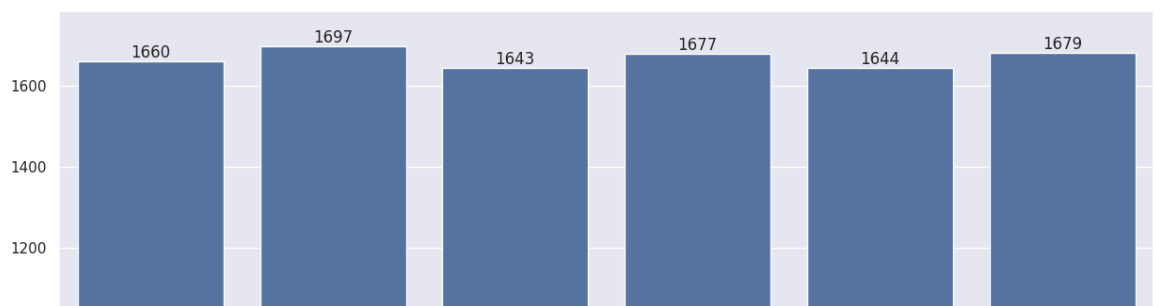
```
perf_study = df.groupby(['attendance'], as_index = False)['previous_score']
sns.barplot(x='attendance', y='previous_score', data = perf_study)
```

<Axes: xlabel='attendance', ylabel='previous_score'>



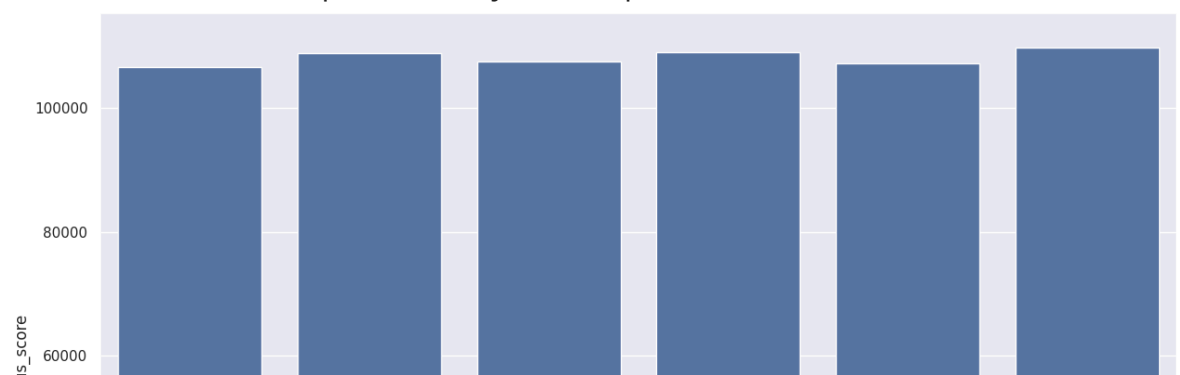
Sleep hours :

```
sns.set(rc = {'figure.figsize' : (15,10)})
ax = sns.countplot(data = df, x = 'sleep_hours')
for bars in ax.containers:
    ax.bar_label(bars)
```



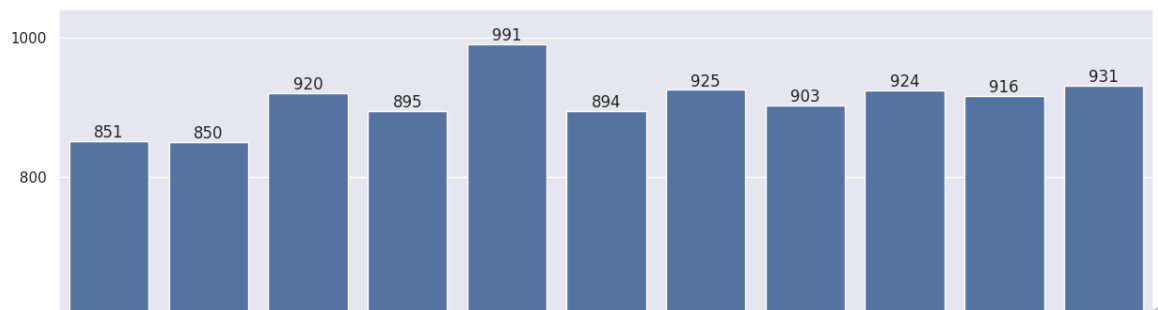
```
perf_study = df.groupby(['sleep_hours'], as_index = False)['previous_score']
sns.barplot(x='sleep_hours', y='previous_score', data = perf_study)
```

<Axes: xlabel='sleep_hours', ylabel='previous_score'>



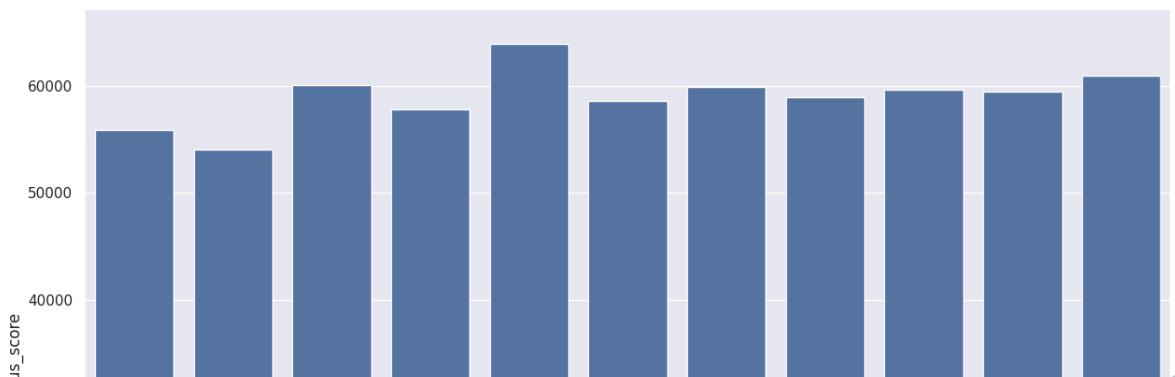
Internet usage :

```
sns.set(rc = {'figure.figsize' : (15,10)})
ax = sns.countplot(data = df, x = 'internet_usage')
for bars in ax.containers:
    ax.bar_label(bars)
```



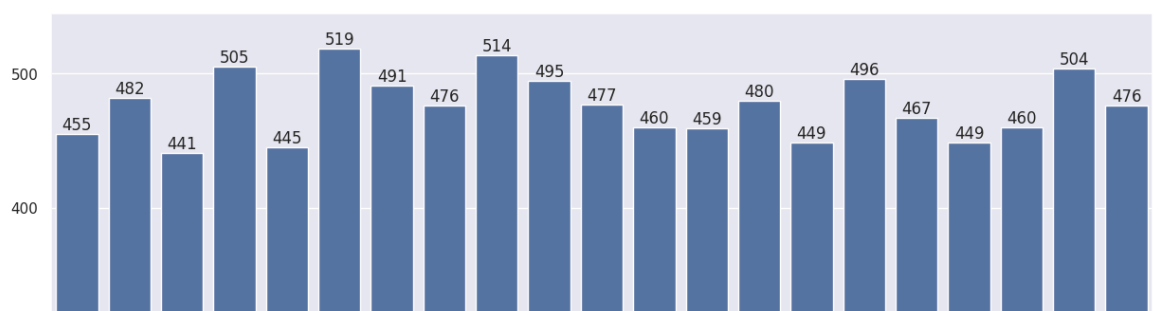
```
perf_study = df.groupby(['internet_usage'], as_index = False)['previous_score']
sns.barplot(x='internet_usage', y='previous_score', data = perf_study)
```

<Axes: xlabel='internet_usage', ylabel='previous_score'>

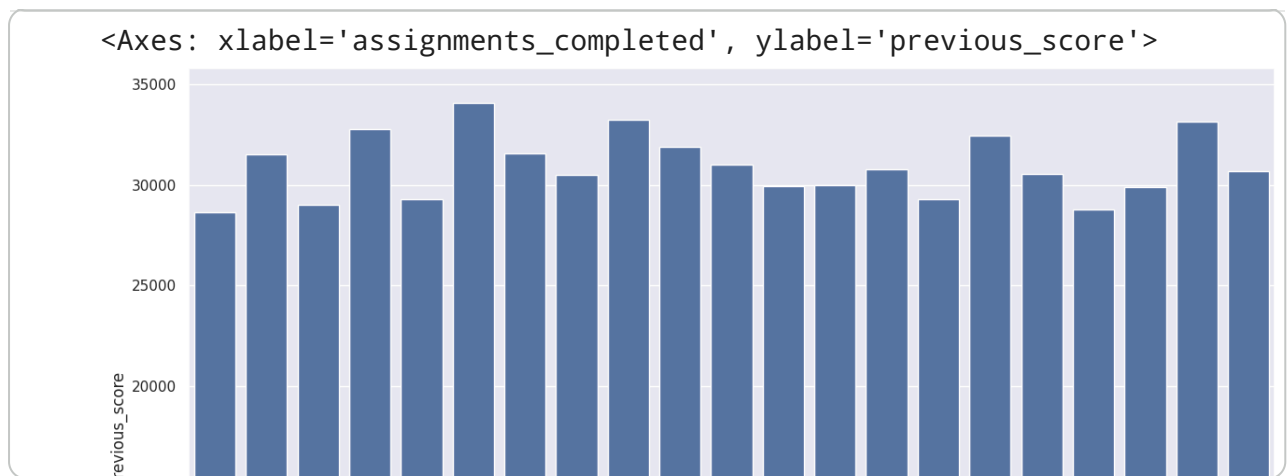


Assignments completed :

```
sns.set(rc = {'figure.figsize' : (15,10)})
ax = sns.countplot(data = df, x = 'assignments_completed')
for bars in ax.containers:
    ax.bar_label(bars)
```



```
'assignments_completed'], as_index = False)['previous_score'].sum().sort_values(
x='assignments_completed', y='previous_score', data = perf_study)
```



Train Test split :

```
from sklearn.model_selection import train_test_split

# Features and target
X = df.drop("exam_score", axis=1) # Replace with your target column name
y = df["exam_score"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Train Model :

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?
LinearRegression()

Make prediction :

```
y_pred = model.predict(X_test)
```

Evaluate Model :

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

MAE: 6.917285717910665
MSE: 77.31965170367612
R2 Score: 0.6634458623051924

Prediction for a new student :

```
new_student = [[5, 90, 7, 2, 8, 75]]  
prediction = model.predict(new_student)  
  
print("Predicted Exam Score:", prediction[0])
```

```
Predicted Exam Score: 92.65120883787014  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:273:  
    warnings.warn(
```

Conclusion :

1. 📊 **Successful Model Development** A Linear Regression model was successfully built to predict students' exam scores using six key features: study hours, attendance, sleep hours, internet usage, assignments completed, and previous scores.
2. 📁 **High-Quality Dataset** The project used a dataset containing 10,000 student records with no missing values, ensuring reliable data analysis and effective model training.
3. 📈 **Meaningful Data Insights** Exploratory Data Analysis (EDA) revealed that study habits, regular attendance, assignment completion, and previous academic performance are among the most influential factors affecting exam scores.
4. 🎯 **Strong Model Performance** The Linear Regression model achieved an R^2 Score of 0.6634, explaining approximately 66.3% of the variation in students' exam scores.
5. 📏 **Model Evaluation** The model delivered promising prediction results with: MAE (Mean Absolute Error): 6.92 MSE (Mean Squared Error): 77.32 These metrics indicate that the model provides reasonably accurate and reliable predictions.
6. 🧠 **Practical Prediction** When tested with a new student's data, the model predicted an exam score of 92.65, demonstrating its real-world applicability in estimating student performance.
7. 🚀 **Future Scope** The prediction accuracy can be further improved by applying advanced machine learning algorithms, feature engineering, hyperparameter tuning, and incorporating additional student-related features.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.